

Accelerating BERT-base training on a single A100

HPC Tools, Lab AI: deliverable 1 (BASELINE)

Nicolás Aller Ponte

30 September 2026

Abstract

We fine-tune BERT-base for question answering on SQuAD v1.1 on one NVIDIA A100 of FinisTerra III and apply the single-GPU optimizations of the course one at a time, profiling each of them. Two epochs take 45.8 minutes in the FP32 baseline and 7.2 minutes once optimized, a $6.4\times$ speedup, while F1 drops by one point (87.97 to 86.97).

1 Setup and method

The model is `google-bert/bert-base-uncased` (109M parameters), trained for two epochs with AdamW (learning rate $3 \cdot 10^{-5}$, 10 % linear warmup, then linear decay) on 88,492 windows of 384 tokens, with PyTorch 2.11 built for CUDA 12.8.

- **Throughput** is measured over 100 steps after a warmup of 30 steps (60 with `torch.compile`), between two `torch.cuda.synchronize()` calls.
- Each configuration runs **three times**; we report the median and the range.
- `torch.profiler` records five steps per configuration. We report the *share* of GPU time per kernel group, which is comparable across configurations; absolute times are not, since the profiled steps are the first ones of each run.
- The baseline is true FP32 (TF32 disabled). Quality is measured with the official SQuAD exact match (EM) and F1 on the full validation set.

2 Running on FinisTerra III

2.1 Resources requested

The work is split into four SLURM jobs (Table 1).

Table 1: Resources requested per job, and actual run time.

Job	GPU	Cores	Memory	Limit	Actual
<code>prepare_data</code>	none	8	4 GB	0.5 h	< 0.5 h
<code>run_improvements</code>	1 A100	32	64 GB	3 h	31 min
<code>run_baseline</code>	1 A100	32	64 GB	3 h	48 min
<code>run_final</code>	1 A100	32	64 GB	2 h	10 min

- **One A100 per job**, as the deliverable requires: the training script uses a single GPU, so more GPUs would sit idle.
- **32 cores**, because FT3 rejects GPU jobs with any other count per A100; eight of them run the DataLoader workers.
- **64 GB of memory**, a generous margin for the tokenized dataset held in memory and the eight worker processes.

- **Time limits** of two to three times the expected duration, so that a slow node does not kill a run, and always below 6 h, which places the jobs in the **short** QoS, the one with the highest priority.
- **Tokenization runs in a CPU-only job** (8 processes, 0.5 GB peak), so no GPU time is spent on it and it is not counted as training time.
- The three GPU jobs are independent, so they were submitted at once on three GPUs: the whole set of measurements took 53 minutes.

2.2 Deployment issues

Table 2: Deployment issues and their solutions.

Issue	Solution
Default PyTorch wheel built for CUDA 13; the driver (570.86.15) supports up to 12.8	Install PyTorch from the CUDA 12.8 index
<code>python/3.10.10</code> not available	<code>cesga/2020 python/3.10.8</code>
GPU job with 4 cores rejected	Exactly 32 cores per A100
<code>\$HOME</code> quota of 20 GB	<code>venv</code> and model cache in <code>\$STORE</code>

3 Results

3.1 One optimization at a time

The optimizations are applied cumulatively, in the order seen in class (Figure 1, Table 3). The three repetitions agree within 1.5 %, except one baseline run 7 % slower, which is why we report the median.

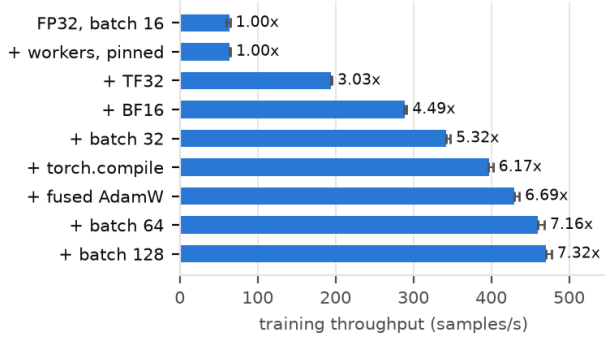


Figure 1: Training throughput after each cumulative optimization: median of three runs, whiskers show the range and labels the speedup over the baseline.

Table 3: Throughput (samples/s, median and range) and peak memory of each step.

Change	Samples/s	Speedup	Mem.
0 FP32, batch 16	64.3 [60.0, 64.7]	1.00×	5.3 GB
1 + workers, pinned	64.4 [64.4, 65.2]	1.00×	5.3 GB
2 + TF32	194.6 [194.6, 195.4]	3.03×	5.3 GB
3 + BF16	289.0 [287.5, 291.5]	4.49×	4.1 GB
4 + batch 32	342.2 [342.2, 347.7]	5.32×	6.6 GB
5 + compile	397.0 [396.1, 402.4]	6.17×	6.0 GB
6 + fused AdamW	430.1 [430.0, 436.2]	6.69×	6.0 GB
7 + batch 64	460.2 [460.0, 468.3]	7.16×	10.4 GB
8 + batch 128	470.4 [470.2, 477.5]	7.32×	19.3 GB

Input pipeline (step 1): no gain. The dataset is tokenized beforehand and kept in memory, so building a batch costs almost nothing next to a 249 ms training step. The GPU never waits for data, and workers and pinned memory have nothing to hide.

Precision (steps 2 and 3): the largest gains. TF32 alone triples throughput without touching the model, because it runs the FP32 matrix products on the Tensor Cores (peak 19.5 to 156 TFLOP/s). BF16 mixed precision adds another 1.5 $\times$: it also halves the bytes moved, and lowers peak memory from 5.3 to 4.1 GB. Since BF16 keeps the FP32 exponent range, no loss scaling is needed.

Batch size (steps 4, 7 and 8): diminishing returns. Larger batches feed larger matrices to the Tensor Cores and spread the fixed cost of each step over more samples. The gain shrinks quickly: +18% for batch 32, +7% for 64 and only +2% for 128, which doubles the memory. The final run therefore uses batch 64.

Compilation and fused optimizer (steps 5 and 6). `torch.compile` adds 16% and fused AdamW another 8%. The profile explains why.

3.2 Where the GPU time goes

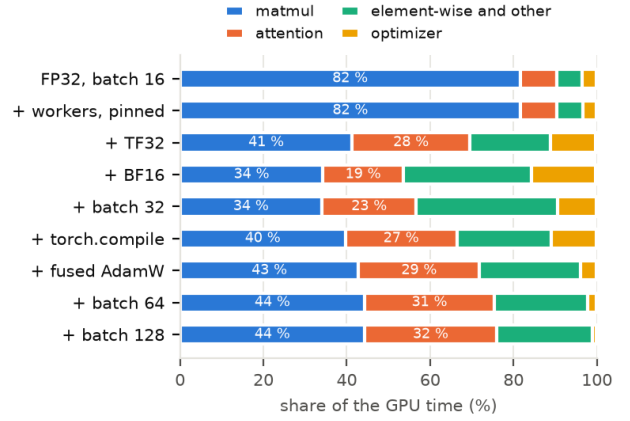


Figure 2: Share of GPU time per kernel group, from the profiler traces. Kernels launched inside the optimizer step count as optimizer; memory copies stay below 0.5%.

In FP32, matrix products take 82% of the GPU time (Figure 2). Once they run on the Tensor Cores, their share falls to about 40%, and kernels that were negligible become visible: speeding up one part makes the rest weigh more, as Amdahl’s law predicts.

Each later optimization targets exactly that remainder. `torch.compile` fuses the element-wise operations (GELU, dropout, residual additions, LayerNorm) into fewer kernels, cutting their share from 34 to 23%. The optimizer then accounts for 11%, because the standard AdamW launches many small kernels; the fused version brings it down to 4%, and larger batches amortize it further (1% at batch 128).

In the final configuration, **attention is the largest cost (32%)**. PyTorch selects the memory-efficient attention kernel rather than FlashAttention, because BERT passes a padding mask, and its backward pass alone takes 23% of the GPU time. This is where the next improvement would have to come from.

3.3 Full training

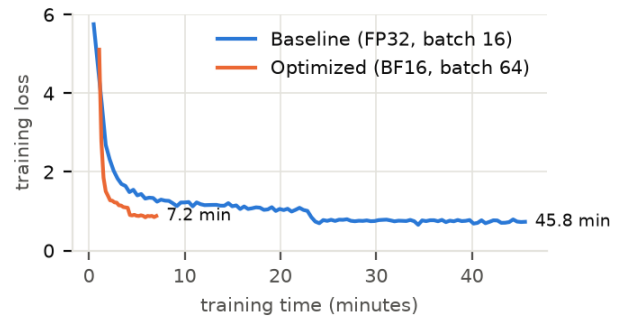


Figure 3: Training loss against wall-clock time for the two full trainings.

Table 4: Full training, two epochs.

	Baseline (step 0)	Optimized (step 7)
Training time	2,747 s (45.8 min)	431 s (7.2 min)
Mean throughput	64.4 samples/s	410.1 samples/s
EM / F1	80.34 / 87.97	79.13 / 86.97
Peak memory	5.3 GB	10.4 GB

Time improves $6.4\times$, less than throughput ($7.3\times$).

The optimized run spends about 56 s outside the steady state, mostly compiling: its curve in Figure 3 starts a minute late. This fixed cost weighs more in a 7-minute run than it would in a longer one. Both curves drop at the start of the second epoch, when the model sees the training data again.

Quality is preserved. The baseline matches the published BERT-base results (80.8 EM, 88.5 F1). The optimized run loses about one point, plausibly because batch 64 makes four times fewer optimizer updates at the same learning rate; tuning it is outside the scope of this deliverable.

Utilization. Counting 204 GFLOP per training sample (Kaplan et al., 2020), the baseline reaches 13 TFLOP/s, 67 % of the FP32 peak, and the optimized run 96 TFLOP/s, 31 % of the BF16 peak: a lower fraction of a peak that is 16 times higher.

4 Conclusions

- Precision is the decisive lever: it accounts for $4.5\times$ of the $7.3\times$ gain in throughput.
- Each optimization exposes the next bottleneck, and profiling after every step shows which one: now it is attention.
- The code was the easy part; most of the work was deploying it correctly on FinisTerra III.